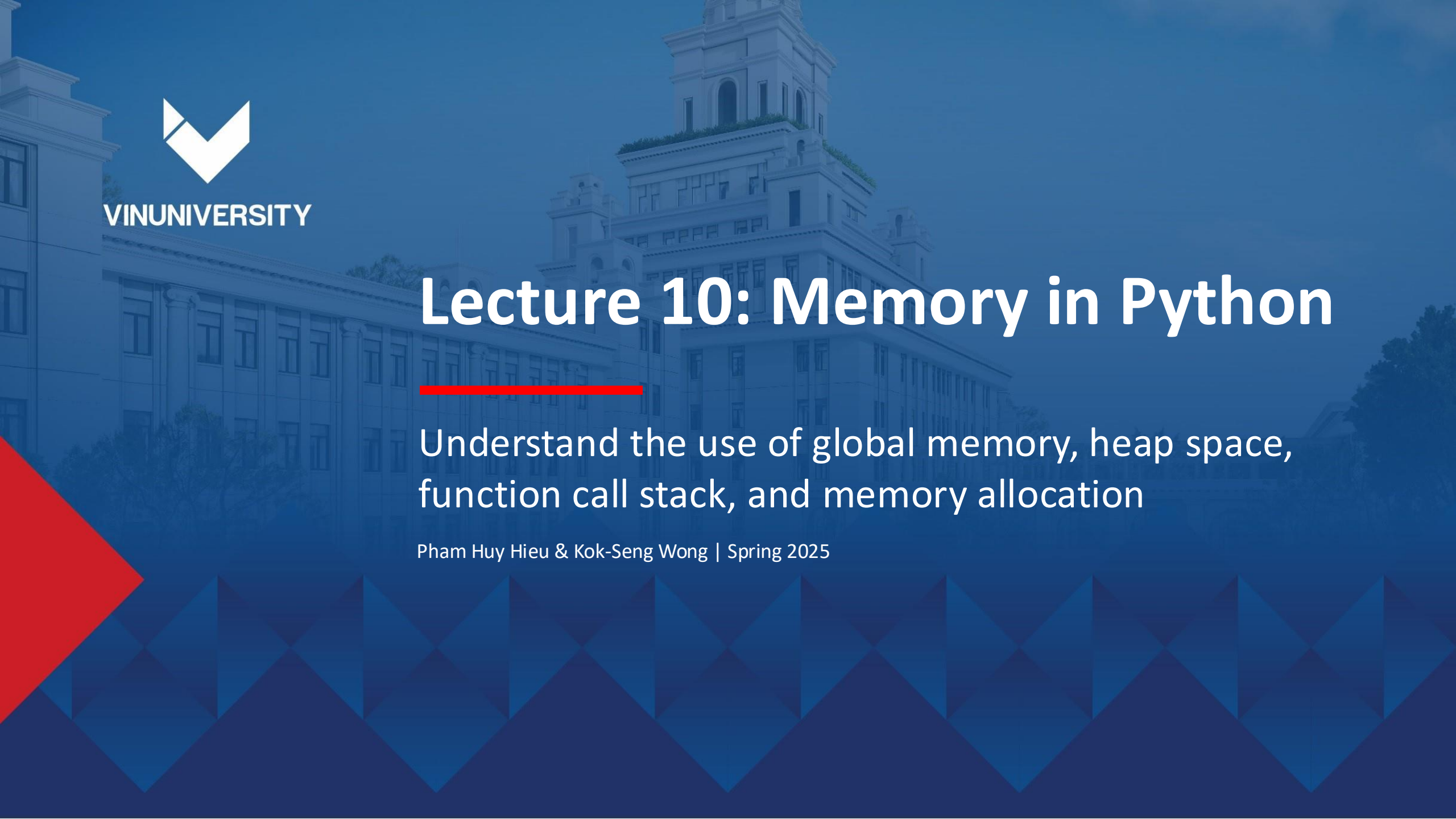


Lecture 10: Memory in Python

Understand the use of global memory, heap space, function call stack, and memory allocation

Pham Huy Hieu & Kok-Seng Wong | Spring 2025

MOTIVATION EXAMPLE



[source]

WHY IS MEMORY MANAGEMENT IMPORTANT?

- Prevents wasted resources and memory leaks
- Avoids invalid memory access errors
- Optimizes performance for large-scale data processing
- Ensures safety in multithreaded programs

Efficient Code = Memory-Efficient Code

[source]

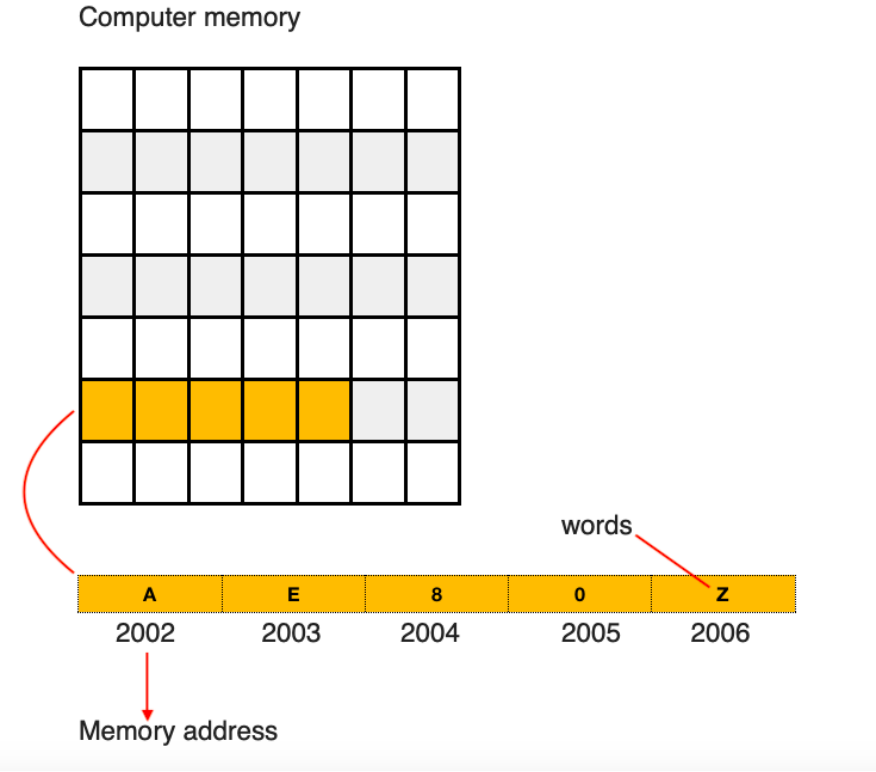
LEARNING OUTCOMES

Upon the completion of this lecture, students will be able to:

- understand the basics about memory management in Python.
- explain why garbage collection (free up the memory) is needed and how Python implements garbage collection.

COMPUTER MEMORY

- We need to use memory on a computer system in order to use any data structure and programs on a computer.
- Computer memory is to store data and program (instructions).



[source]

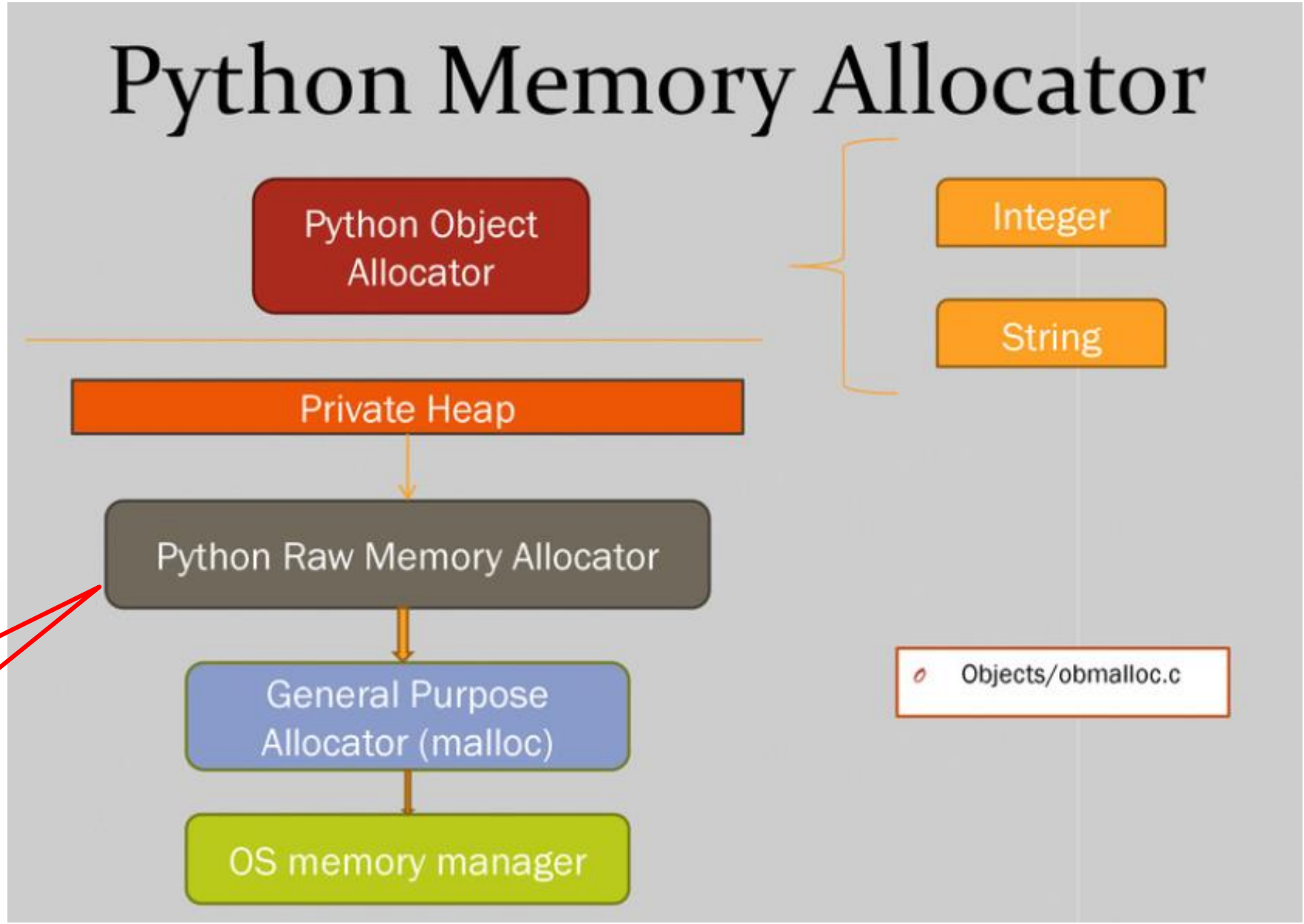
MEMORY MANAGEMENT

Efficient Code = Memory-Efficient Code

- Memory Allocation: allocation of a space (or a block) in the computer memory to a program.
- Garbage Collector: automatically handles memory allocation and de-allocation
- Memory management is the process of efficiently allocating, de-allocating, coordinating memory so that all the different processes run smoothly and optimally access different resources.

MEMORY MANAGEMENT IN PYTHON

- Private heap: contains all Python objects and data structures.
- The interpreter performs the Python heap management, where the user has no control over it!
- The memory is further divided into:
  - Heap Memory (objects and instance variables)
  - Stack Memory (methods and variables)

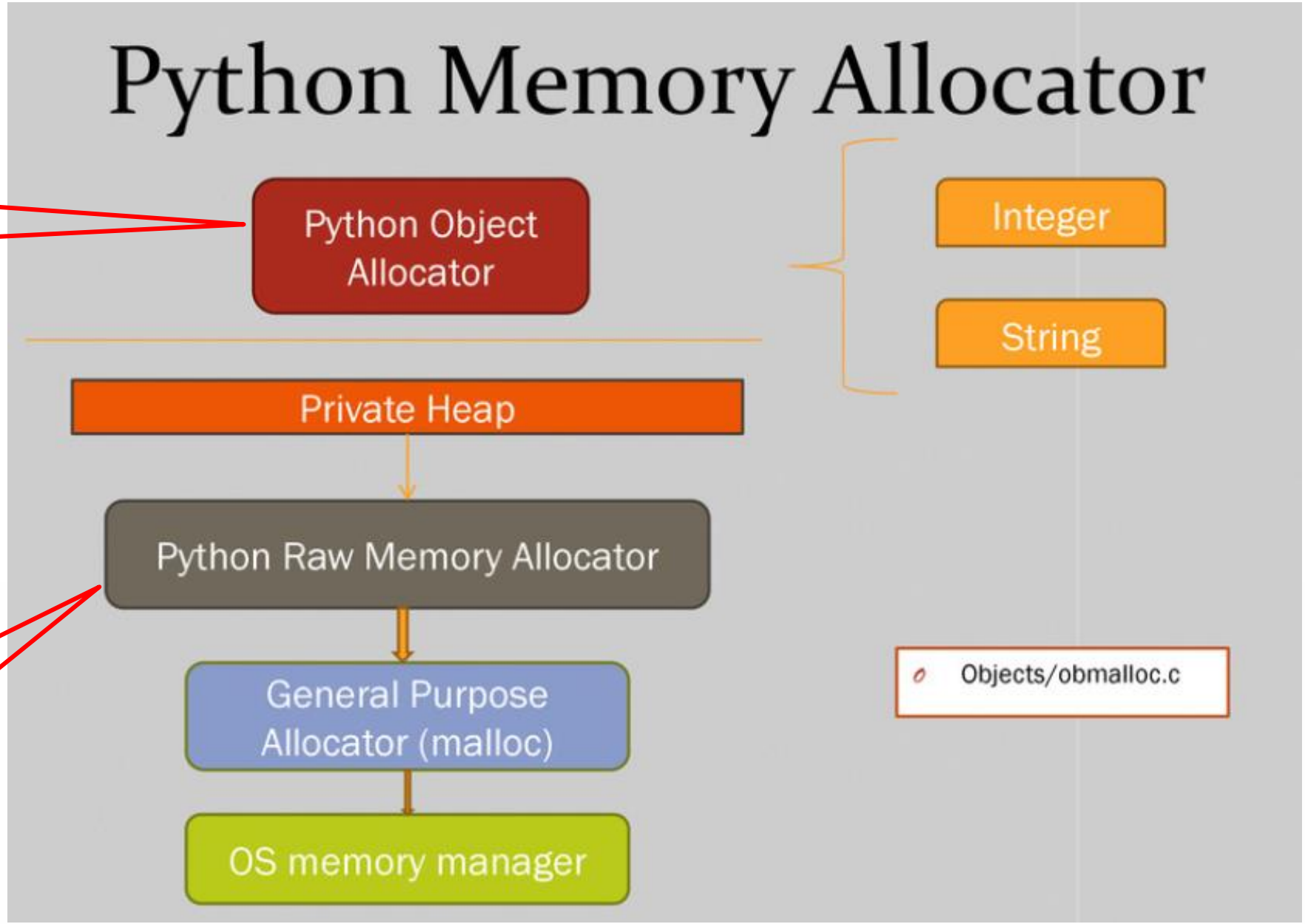


To ensure that there is enough room in the private heap for storing all Python-related data

[source]

Implement distinct memory management policies adapted to the peculiarities of every object type.

To ensure that there is enough room in the private heap for storing all Python-related data



[source]

GARBAGE COLLECTION

Process in which the interpreter frees up the memory (when not in use) to make the available memory for other objects.

REFERENCE AND REFERENCE COUNT

What is a reference? A **reference** is a link or pointer to an object stored in memory. Every time you assign an object to a variable, Python creates a *reference* to that object.

```
import sys
v1 = 10
v2 = 10
v3 = v2

r1 = sys.getrefcount(v1)
r2 = sys.getrefcount(v2)
r3 = sys.getrefcount(v3)
print(r1, r2, r3)


3 3 3
```

Pointing to the same memory address

```
import sys
v1 = 10
v2 = 10
v3 = v2
v4 = v1

r1 = sys.getrefcount(v1)
r2 = sys.getrefcount(v2)
r3 = sys.getrefcount(v3)
r4 = sys.getrefcount(v4)
print(r1, r2, r3, r4)
4 4 4 4
```

Reference count +1 when adding a new variable which is equal to 10

REFERENCE AND REFERENCE COUNT

- Every object keeps a reference counter, which tells how many variables (or references) are currently pointing to that object.

- When a new reference to the object is created, counter increases.

- When a reference is deleted or goes out of scope, counter decreases.

- If the counter reaches zero, it means no variable is using the object anymore, so Python automatically deallocates (frees) that memory.

GARBAGE COLLECTION

Garbage collection: It is a process in which Python automatically frees memory occupied by objects that are no longer in use.

- If an object has no references pointing to it (i.e., nothing is using it), garbage collector removes it from memory.

- This ensures that unused memory can be reused for new objects.

GLOBAL SPACE VS. HEAP SPACE

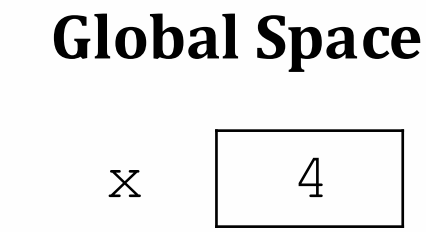
- **Heap Space** is the region of memory where objects and data structures are stored during program execution.
- **Global Space** refers to the area where variable names and function names are stored when a Python script runs.

Aspect	Global Space	Heap Space
Storage Location	Global variables, constants, and functions.	Dynamically allocated objects (e.g., class instances, lists).
Lifetime	Exists for the lifetime of the program.	Exists as long as there are references to it; managed by garbage collection.
Scope	Can be accessed globally in the module or across modules.	Objects in heap space can be accessed by references, which can be global or local.
Access	Direct access to global variables and functions.	Access via references (e.g., pointers or variable names).
Memory Management	Handled by Python for global variables.	Managed dynamically through Python's memory manager and garbage collector.



GLOBAL SPACE

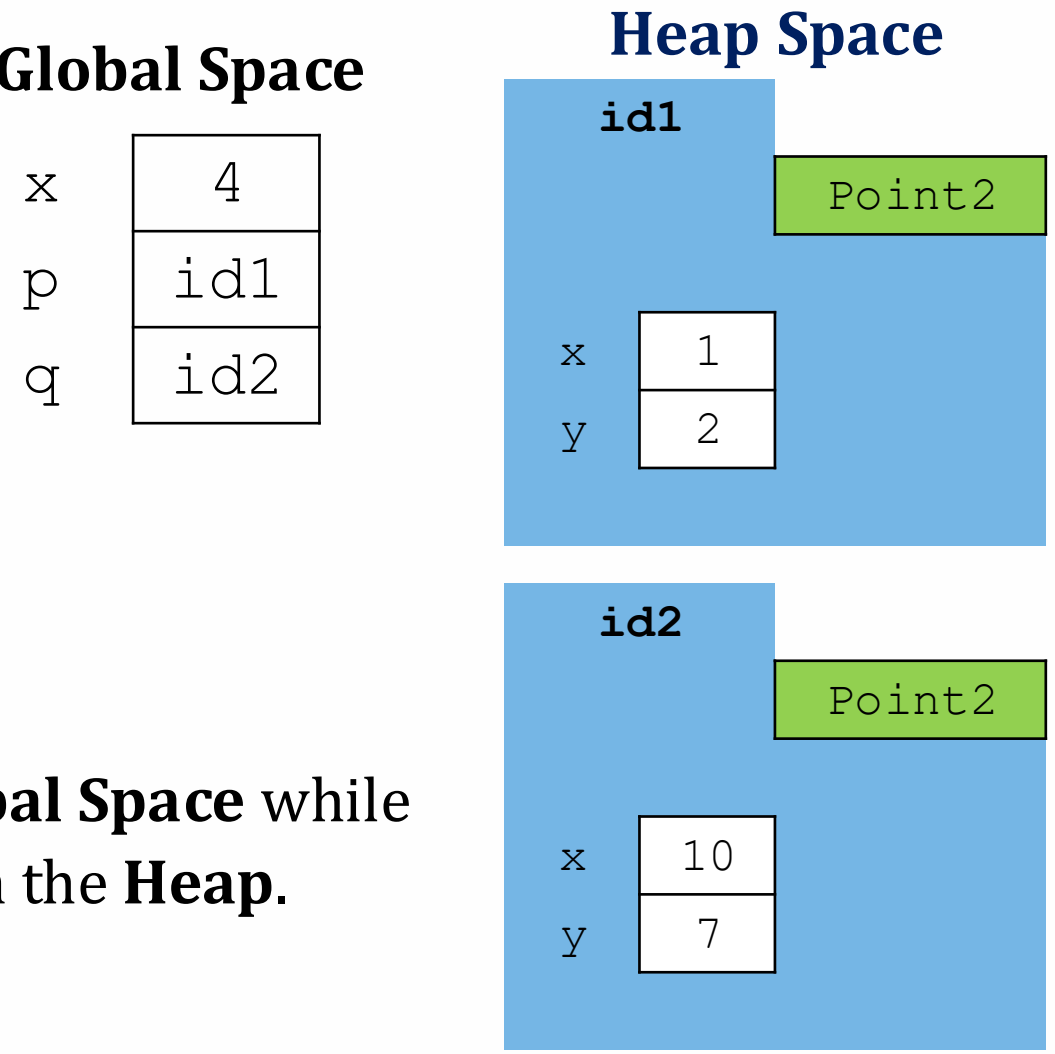
- Used to store global variables
- Lasts until you quit Python



HEAP SPACE

- Where "folders" are stored
- Have to access indirectly

```
x = 4
p = shape.Point2(1,2)
q = shape.Point2(10,7)
```



p and **q** live in **Global Space** while their folders live on the **Heap**.

CALL FRAME: WHEN CALLING A FUNCTION

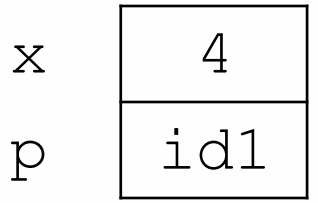
- **Call frame** is the place that stores the parameters and local variables of each function call.
- Deleted when call done!

```
def adjust_x (pt,n):
    pt.x = pt.x + n

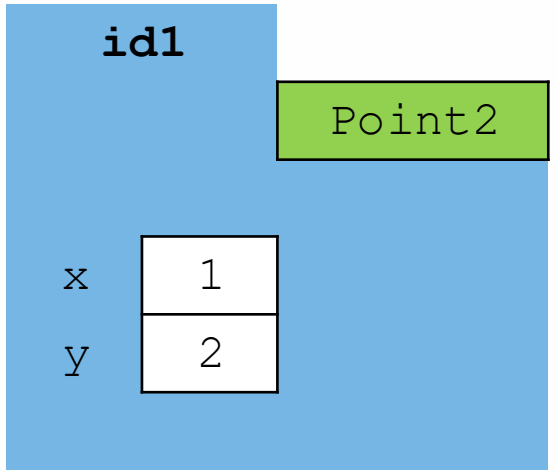
x = 4
p = shape.Point2(1,2)
adjust_x(p,x)
```

1

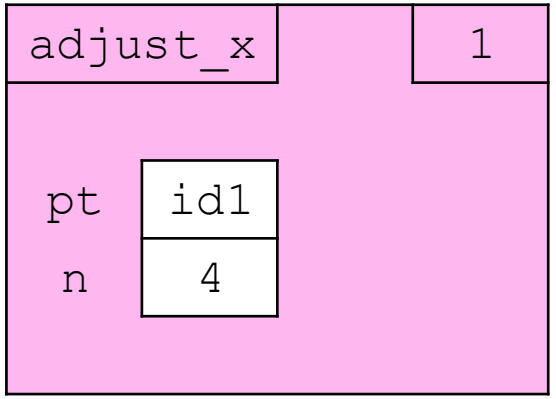
Global Space



Heap Space



Call Frame



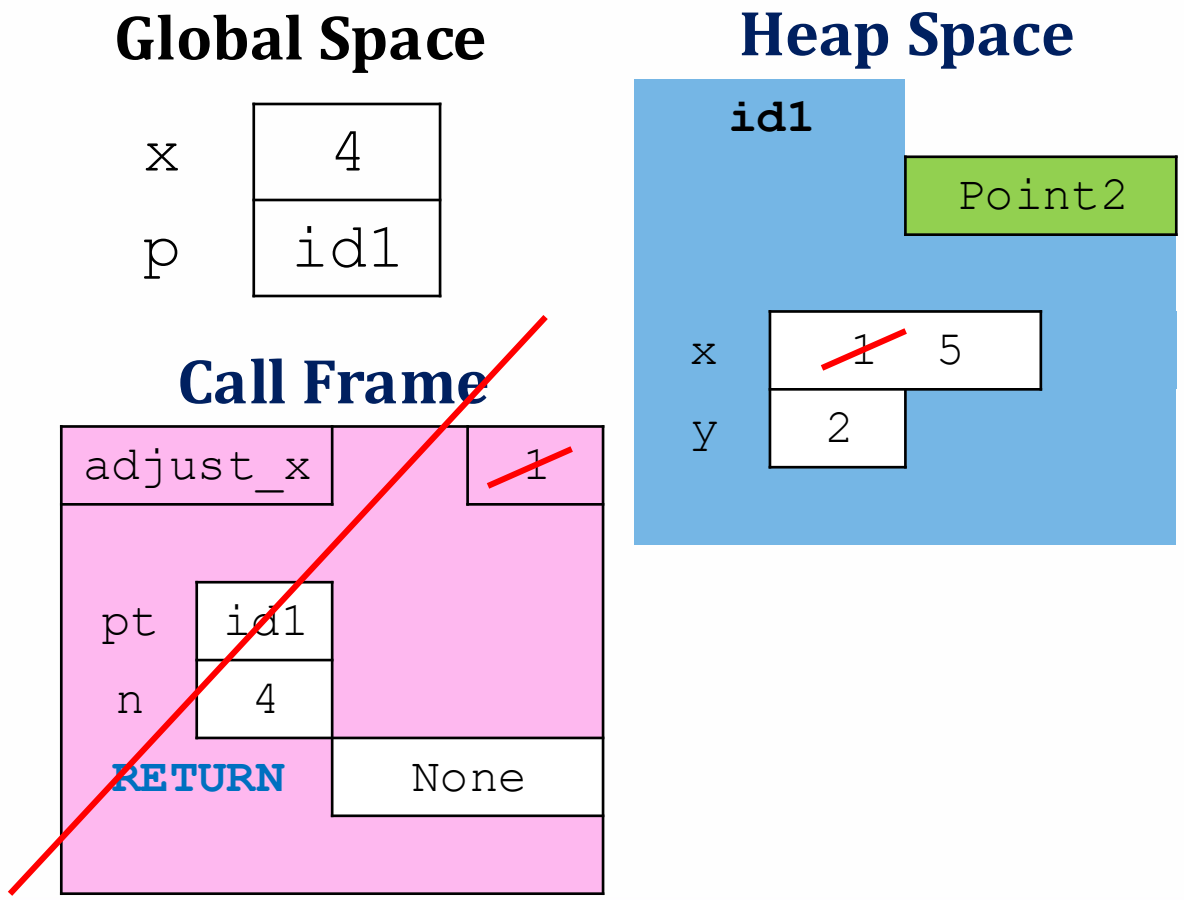
CALL FRAME: WHEN CALLING A FUNCTION

- **Call frame** is the place that stores the parameters and local variables of each function call.
- Deleted when call done!

```
def adjust_x (pt,n):
    pt.x = pt.x + n

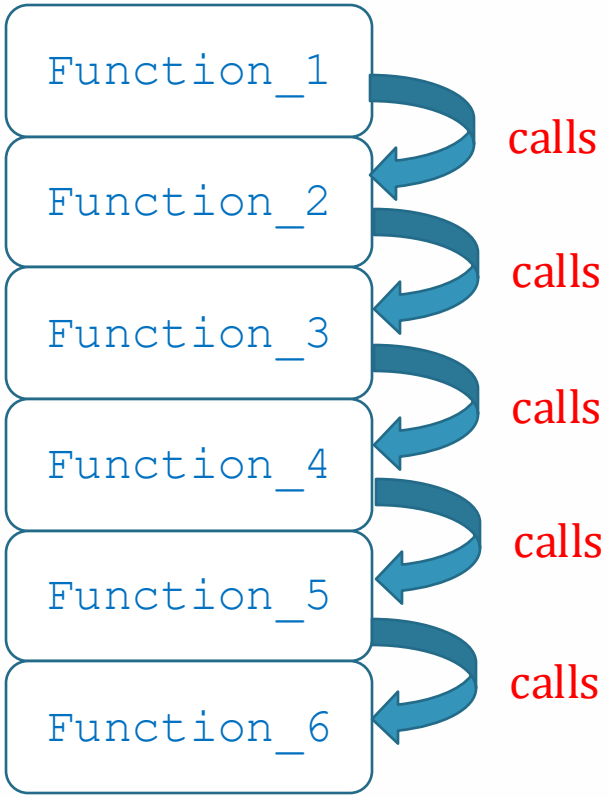
x = 4
p = shape.Point2(1,2)
adjust_x(p,x)
```

1



THE CALL STACK

- The set of function frames drawn in call order.
- Functions frames are "stacked": cannot remove one above without removing one below.
- Python must keep the entire stack in memory: error if it cannot hold stack ("stack overflow").



CALL STACK – EXAMPLE

A call stack is a mechanism for keeping track of the active subroutines of a computer program. It is a LIFO (Last-In, First-Out) data structure, meaning the last function added to the stack is the first one to be removed.

Python

```python
def multiply(a, b):
    return a * b

def square(x):
    return multiply(x, x)

def print_square(num):
    result = square(num)
    print(f"The square of {num} is {result}")

print_square(5)
```

How the call stack would behave during the execution of print_square(5):

- Initial State: The call stack is empty.
- print_square(5) is called. A stack frame for print_square(5) is pushed onto the stack.
- Call Stack: [print_square]

CALL STACK – EXAMPLE

A call stack is a mechanism for keeping track of the active subroutines of a computer program. It is a LIFO (Last-In, First-Out) data structure, meaning the last function added to the stack is the first one to be removed.

Python

```python
def multiply(a, b):
    return a * b

def square(x):
    return multiply(x, x)

def print_square(num):
    result = square(num)
    print(f"The square of {num} is {result}")

print_square(5)
```

How the call stack would behave during the execution of print_square(5):

- Initial State: The call stack is empty.
- print_square(5) is called. A stack frame for print_square(5) is pushed onto the stack.
- Call Stack: [print_square]
- square(num) is called from within print_square(). A stack frame for square is pushed onto the stack.
- Call Stack: [print_square, square]

CALL STACK – EXAMPLE

A call stack is a mechanism for keeping track of the active subroutines of a computer program. It is a LIFO (Last-In, First-Out) data structure, meaning the last function added to the stack is the first one to be removed.

Python

```python
def multiply(a, b):
    return a * b

def square(x):
    return multiply(x, x)

def print_square(num):
    result = square(num)
    print(f"The square of {num} is {result}")

print_square(5)
```

How the call stack would behave during the execution of print_square(5):

- Initial State: The call stack is empty.
- print_square(5) is called. A stack frame for print_square(5) is pushed onto the stack.
- Call Stack: [print_square]
- square(num) is called from within print_square(). A stack frame for square is pushed onto the stack.
- Call Stack: [print_square, square]
- multiply(x, x) is called from within square. A stack frame for multiply is pushed onto the stack.
- Call Stack: [print_square, square, multiply]

CALL STACK – EXAMPLE

A call stack is a mechanism for keeping track of the active subroutines of a computer program. It is a LIFO (Last-In, First-Out) data structure, meaning the last function added to the stack is the first one to be removed.

Python

```python
def multiply(a, b):
    return a * b

def square(x):
    return multiply(x, x)

def print_square(num):
    result = square(num)
    print(f"The square of {num} is {result}")

print_square(5)
```

How the call stack would behave during the execution of print_square(5):

- multiply() returns a * b: The multiply stack frame is popped off the stack.
- Call Stack: [print_square, square]
- square(x) returns the result of multiply (25): The square stack frame is popped off the stack.
- Call Stack: [print_square]
- print_square(num) finishes execution: The print_square() stack frame is popped off the stack.
- Call Stack: [] (empty)

MODULES AND GLOBAL SPACE

Import

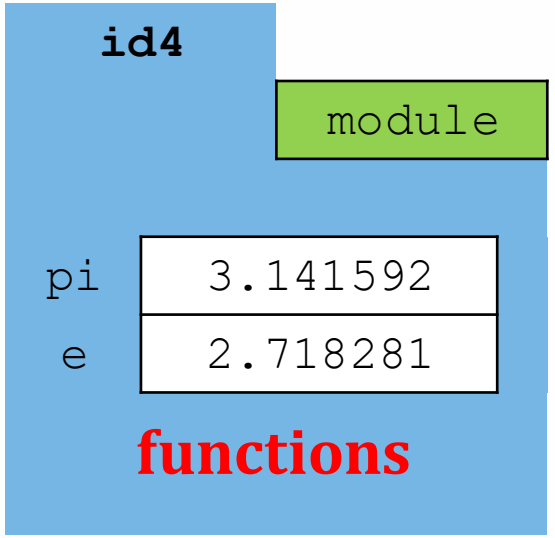
- Creates a global variable (same name as module)
- Puts variables, functions of module in a folder (Heap space)
- Puts folder id in the global variable

```
>>> import math
```

Global Space

math	id4

Heap Space



MODULES VS OBJECTS

```
>>> import math
>>> math.pi
```

```
>>> p =
shapes.Point3(5, 2, 3)
>>> pi.x
```

Global Space

math	id4
p	id5

Heap Space

id4

Math module

pi	3.141592
e	2.718281

functions

id5

Point3

x	5
y	2
z	3

FUNCTIONS AND GLOBAL SPACE

A function definition

- Creates a global variable (same name as function)
- Creates a folder for body
- Puts folder id in the global variable

```
def larger (a,b):
    if a > b:
        return a
    else:
        return b

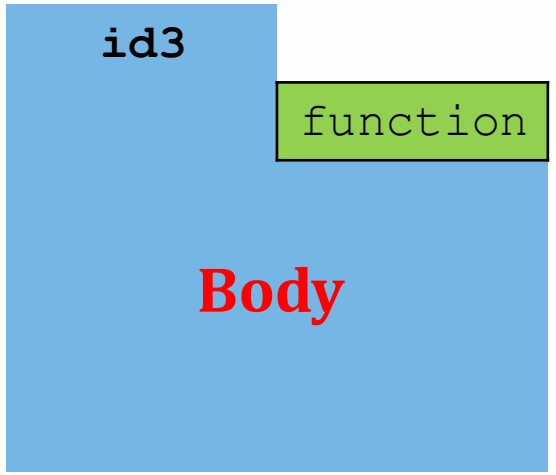
x = 10
y = 20
larger(x, y)
```

Body

Global Space

larger	id3
x	10
y	20

Heap Space



THANK YOU!